



INF1103 Programming Fundamentals with DevOps

Week 1: Expectations and Foundations

Outline



Module Introduction



Learning Pathway Overview



Module Expectations



Development Environment Setup



Python Programming Basics



► Nisha Jain(Week 1-6)

► Email:

nisha.jain@singaporetech.edu.sg

► Oran Zane Devily (Week 8-13)

► Email:

oran.devily@singaporetech.edu.sg



Who will teach

Competency based Learning

Mastery Framework

- This module follows a **Competency-Based** approach. You are not just earning marks; you are demonstrating professional mastery.
- **C1:** Develop structured software applications using procedural programming principles.
- **C2:** Develop modular and reusable software applications using object-oriented programming principles.
- **C3:** Apply DevOps tools and practices to manage, integrate, and deploy software applications.

The “D” Grade Rule

- To pass INF1103, you **must** achieve at least a “D” grade in each individual competency.

Note: Failing one competency means failing the unit. You have up to 3 attempts across 2 trimesters to clear a deficiency.

Outline



Module Introduction



Learning Pathway Overview



Module Expectations

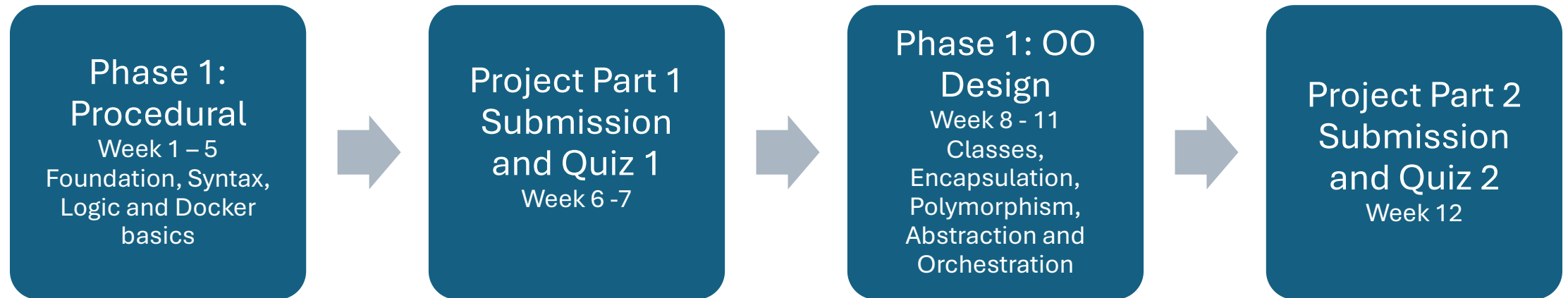


Development Environment Setup



Python Programming Basics

12 Week Road Map



Competency Assessment

No	Specific Competency	Assessment Task(s)	Weightage %
1	Procedural Project Development and Management (C1 + C3)	Project (Development Managed with Dev Ops) Weekly Lab Activities	30%
2	Object Oriented Project Development and Management (C2 + C3)	Project (Development Managed with Dev Ops) Weekly Lab Activities	30%
3	Procedural Programming Theory Assessment (C1)	Theory Quiz	20%
4	Object Oriented Programming Theory Assessment (C2)	Theory Quiz	20%

Schedule for First Half



Weekly Labs Assignments



Released every Week on the Lab day



You will have one week to complete



Weekly Lab Assignments

- Hands on learning
- Use the lab time to clear any doubts related to the topics of the week
- Ask questions as many as you want
- Team discussions to work on the project



Team Project

- Teams (5-6 students)
- Already assigned in your respective lab group (check your xsite)
- No change requests for team change will be considered
- Details will be shared next week...

Outline



Module Introduction



Learning Pathway Overview



Module Expectations



Development Environment Setup



Python Programming Basics

Why learn Programming?

- Solve real-world problems through **logical thinking**
- Break complex tasks into smaller manageable steps
- Build software, automate processes and create value
- The goal is **not memorizing** syntax but learning how to think

What Industry is looking for?

- Programming fundamentals and **problem-solving skills**
- Logical reasoning and computational thinking
- Ability to debug and troubleshoot systematically
- Communication, teamwork and continuous learning
- Using **AI effectively** while knowing what the code actually does

About using ChatGPT and friends...

- You must be proficient in using Gen AI tools
- But you still have to have **strong programming foundations** to understand, debug and develop

"AI will make coders 10× more productive, not replace them."

Sam Altman,
CEO of OpenAI

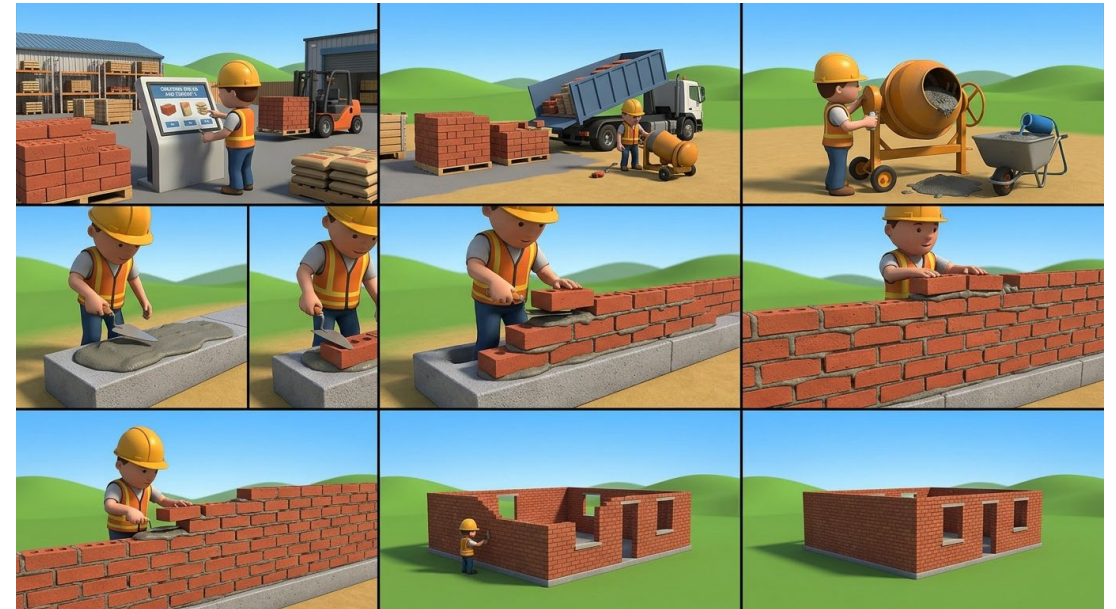
kiboshib.com



AI can write the code.
Can it think?

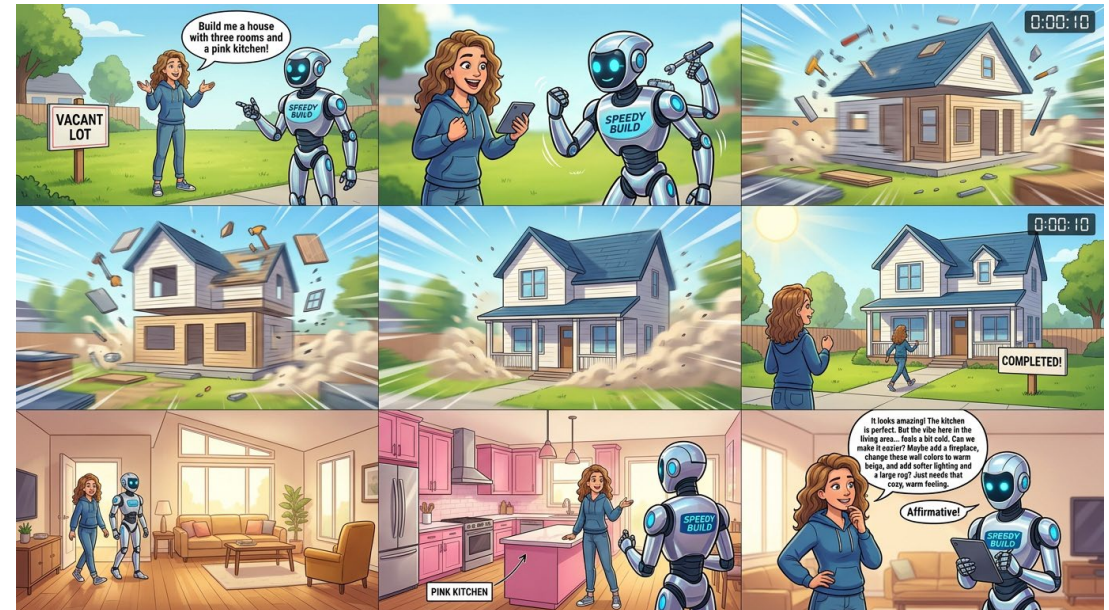
Traditional Coding: Home Builder

- Buying from scratch and building the whole thing up
- It takes months to learn and a long time to build, but you know exactly what is inside every wall.



Vibe Coding : Home Builder

- hiring a hyper-fast, automated robotic contractor
- "Build me a modern 3-bedroom house with a pink kitchen."
- You didn't lay a single brick; you just steered the contractor.



Vibe coding: Usage



Instant Prototyping: use genAI tools to describe the app and have a functional prototype running in minutes



Building "Casual" Apps: even non-technical people use it



Shifting from "Writer" to "Orchestrator": use AI to write the code blocks so they can spend their brainpower on high-level architecture and system design.

Vibe coding: The Dangers



Vulnerabilities: AI models copied from flawed data (behind wall electrical wires in water pipes)



Bug fixing: how to solve complex bugs (specific plumbing issue)



No broader Context: integration with existing API (build house on earthquake prone area)



Your Advantage

- Everyone has access to AI tools
- Strong fundamentals create long-term career value
- Curiosity, adaptability and logical thinking differentiate engineers
- Ask: “Do I understand why this solution works?”

What should you do?

- Start with Basic Programming Knowledge
- **Practise** enough to solve complex problems
- Break complex problems into smaller modules
 - You can use AI **assistance**, but with an intent to learn
- Do it yourself
 - Plan-> implement->test->repeat

How can AI help!!

Pre Processing (good starting line)

- Brainstorming ideas
- Designing functionality
- Initial workflows

Implementation

- Simple code snippets
- Debugging errors

Deliverables

- Report Writing: grammar check



What should
you expect?
Feedback
(many forms)

- Each week you can get more **focused feedback on your code** based on the questions you ask
- We will be using **TeamAid** for regular feedbacks (more next week)

What can you expect?

- Fundamental skills in programming (Python) and DevOps
- Don't worry even if you are new to all of this
- Even if you are expert in programming, hope you still learn and develop something new
- One mantra: Practise, Practise and more Practise.....

What we expect?

- **Plagiarism** is unacceptable: you should not copy paste from any other student's work or allow any other student to copy your work
- Using AI is ok; but donot use it as a shortcut to do the work
- Aim to learn and improve your abilities; that is what will help you survive in the industry



Takeaways!!

- AI is prone to make mistakes
- Use AI as a learning aid but not a replacement
- Keep the intent clear, you are here to learn
- Don't stop thinking

Outline



Module Introduction



Learning Pathway Overview



Module Expectations



Development Environment Setup



Python Programming Basics

Building Blocks

- Python as Core Language
- Module Structure and Progression
- DevOps Integration
- Hands-On Learning Emphasis





Collaboration and Git

The Shared Sketchbook

- If Git is your **code** time machine, GitHub is the **studio**.
- Writing software individually is simple, but combining work with team members often leads to overwriting each other's changes.
- GitHub operates like Google Docs for code. It allows multiple developers to safely branch off, draft features asynchronously, comment on changes, and systematically merge them together once approved.

GitHub

GitHub (The Written Record)

- Manages Text and History
 - **What it stores:** Source code files, markdown instructions, and the revision history of who changed what, when, and why.
 - **Layman Analogy:** A collaborative architect's sketchbook. It organizes, displays, and versions the floorplans. It cannot build the house; it only holds the plans.
-



Takeaway!!

- Standardizing improves team collaboration and consistency in software development projects.

Outline



Module Introduction



Learning Pathway Overview



Module Expectations



Development Environment Setup



Python Programming Basics

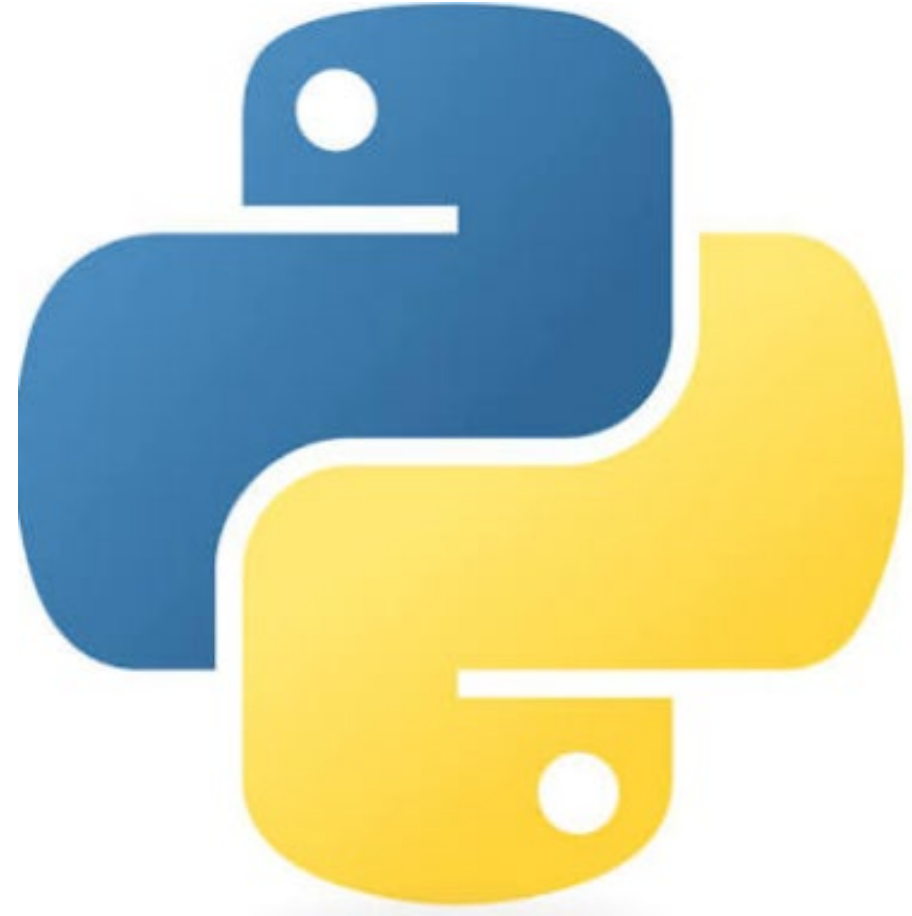


What is an IDE?

- IDE – Integrated Development Environment
- Write, run and debug code
- Reduces errors and improves productivity by combining file management, extensions, and testing support.
- VS Code or Eclipse or any other

Why Python?

- High level, interpreted, general purpose language
- Simple and powerful
- Versatility



Python in industry!!

- Netflix recommending movies
- Spotify recommending songs
- Face recognition on smartphones
- AI chatbots

- Smart homes
- Fitness trackers
- Environmental monitoring systems

- AI-assisted diagnosis
- Health monitoring apps
- Wearable devices

- Credit card fraud detection
- Mobile banking apps
- Investment analytics

Your First Python Program

- Write one program to print "Hello World!" in Python
 - `print ("Hello World!")`
 - Or
 - `print ('Hello World!')`
- `print()`
 - to print something to the screen
 - Using either " " or ' ' around what you want to print
 - Further Reading
 - The arguments of `print()` function

Knowing Python

- Single Quote and Double Quote can generally be used interchangeably.
 - `print('Hello, World! ')`
 - `print("Hello, World! ")`
 - `print('He said, "Python is awesome! "')`
 - `print("It's a beautiful day! ")`

Knowing Python

- What if I want to print
 - It's a "beautiful" day.
- Use a backslash ('\') to escape quotes inside the string

```
print('It\'s a "beautiful" day! ')
```

```
print("It's a \"beautiful\" day! ")
```
- Escape characters

\n	Newline
\t	Tab
\\	backslash

Knowing Python

- Raw strings
 - Prefixing a string with r or R makes it a raw string, where backslashes are not treated as escape characters
 - `print(r'it\n1\t2')`
 - `print('it\n1\t2')`
 - `print(r"C:\Users\Name")`
 - `print("C:\Users\Name")`

Knowing Python

- The language is case sensitive
 - `print` $\neq$ `Print` $\neq$ `PRINT`
- Indentation matters
 - Some development tools would make automatic indentation
 - Be careful about indentation
 - In nested `if`, `for`, `while`

Write comments

- Comments
 - People may read your code
 - Guide your thinking
 - In a single line
 - Using the symbol **#**
 - For multiple lines
 - Using three quotation marks `''' '''`
 - Docstring

```
#this is a comment  
print('Hello \"World\"')
```

Variables and Assignment Statements

- Variables
 - `<variable name> = expression`
 - `print('Hello, World! ')`
 - `print("Hello, World! ")`
 - `print('He said, "Python is awesome! "')`
 - `print("It's a beautiful day! ")`
- `print('Nisha')`
- `firstName = 'Nisha'` ←
- `print(firstName)`

Assignment statement

Variables as names

- Variables
 - `<variable name> = expression`
 - A name
 - We can use the name in following expressions
 - **Variable references**
 - Values can be reset to new ones

```
firstName = 'Nisha'  
lastName = 'Jain'  
fullName = firstName + ' ' + lastName  
print(fullName)
```

Nisha Jain

```
firstName = 'Neha'  
fullName = firstName + ' ' + lastName  
print(fullName)
```

Neha Jain

Variable Naming Rules

- Variables must **start with a letter or an underscore _**
- Subsequent characters can be **letters, numbers, or underscores**
- Variable names are **case-sensitive**
- Keywords **cannot** be used as variable names:
 - Examples of keywords: **if, else, for, while, class, def, return, try, except**, etc.
- Which one is valid?
 - **_myvar, Var1, 1var, -var, return, good**

Variable Naming Rules

- Variables must **start with a letter or an underscore** _
- Subsequent characters can be **letters, numbers, or underscores**
- Variable names are **case-sensitive**
- Keywords **cannot** be used as variable names:
 - Examples of keywords: **if, else, for, while, class, def, return, try, except**, etc.
- Which one is valid?
 - `_myvar`, `Var1`, `1var`, `-var`, `return`, `good`

Variable Naming Rules

- Use lowercase letters and underscores for variable names (snake_case):
 - Example: `my_variable`, `user_name`, `total_sum`
- Constants (values that do not change) are usually written in all uppercase with underscores (UPPERCASE_WITH_UNDERSCORES):
 - Example: `PI = 3.14159`, `MAX_LIMIT = 100`
- Use meaningful variable names:
 - Choose names that clearly describe the variable's purpose, e.g., `total_sum` instead of `ts`.
 - Avoid single-character variable names, unless in loops (e.g., `i`, `j`, `k`):

Variable Naming Rules

- Follow the PEP 8 style guide:
 - PEP 8 is the Python community's style guide, recommending consistent and readable naming practices.
 - <https://peps.python.org/pep-0008/>
- Python 3 supports Unicode characters, allowing variable names in non-Latin scripts like Chinese or Japanese, but it's best to maintain consistency and readability.

Variables as Memory Location

- Variables
 - <variable name > = expression
 - A variable is a **reserved memory location** used to **store values**
 - How can I know the memory address of a variable?
 - `id()`

```
firstName = 'Nisha'  
print(firstName)  
print(id(firstName))
```

```
Nisha  
2343117634656
```



Multiple Assignment

- Practice
 - How can we swap the values of two variables?
 - first = 10, second = 20
 - first = 20, second = 10

Variable types

- Variables
 - Name
 - Id
 - Type
- How to know a variable's type?
- `type()`

Variable types

Type	Samples
int	8,12,1024
float	2.3, 3.1415926
bool	True, False
str	'Hello, World! ', '3.1415926'
None	None
List	
Tuple	
Set	
Dictionary	
byte	

Type Casting and Dynamic Typing

- Type casting is when you explicitly convert a value from one type to another.
- Dynamic typing means the variable's type is determined automatically at runtime.

- Type casting

```
x= 10  
print(x,type(x))  
y = str(x)  
print(y,type(y))
```

```
10 <class 'int'>  
10 <class 'str'>
```

- Dynamic Typing

```
x= 10  
print(x,type(x))  
x = 'string'  
print(x,type(x))
```

```
10 <class 'int'>  
10 <class 'str'>
```



Type Casting

- Numbers to string
- String to numbers
- Numeric precision adjustment
 - Int and float

Arithmetic Operators

Operator	Name	Example
+	Addition	
-	Subtraction	
*	Multiplication	
/	Division Always return float	10/0 ZeroDivisionError 4/2=2.0 5/2=2.5
//	Floor Division	13.9//2 = 6.0
%	Modulus	11%3 = 2; 11.0%3.0 = 2.0
**	Exponent	2**4 = 16
+=	Augmented Addition Operator	a+=1 equals to a = a+1
-=		
*=		
...		



Takeaway!!

- Python programs run instructions one after another, showing clear flow and execution order.
- Indentation defines code blocks in Python, emphasizing readability and structure over symbols.
- Comments are used to explain code purpose, improving clarity and collaboration among developers.
- Consistent syntax, correct spelling, and case sensitivity are vital for error-free Python code.
- Programming is about **problem-solving**, **practice**, and **logical** thinking.



Lab this week!!

- Meet your **lab-in-charges** and student helpers
- Get acquainted with your team members
- Install IDE, git and python on your laptops
- Complete Weekly Lab